Session 4 - Coalesced memory access and device (GPU) memory types
Geavanceerde computerarchitectuur: labosessie

(B-KUL-JLI1QS)

Elias **Neels**, Wout Batselé

March 12, 2026



Year: 2026

Class: MELEC

Instructor:

Contents

1 Context **2**

2 Methodology **2**
2.0.1 Where should we put `__syncthreads()` ? 2
2.0.2 Coalescing 2
2.0.3 Memory types 3

3 Observations/results **4**

4 Conclusion **6**

A Overige grafieken **7**

B Code **7**
B.1 Part 1 7
B.2 Part 2 11

1 Context

In dit labo ligt de focus op het optimaliseren van geheugentoegang op de GPU. We onderzoeken hoe *coalesced memory access* de bandbreedte maximaliseert door geheugenverzoeken van een warp te combineren. Daarnaast evalueren we de impact van verschillende GPU-geheugentypes (Global, Shared en Constant) op de prestaties.

2 Methodology

2.0.1 Where should we put `__syncthreads()` ?

Threads migrate data from global to shared memory.

1. (on CPU) Copy data to global memory and run the kernel
2. (on GPU, per thread) Migrate one or more elements to shared memory
3. Process the data in shared memory
4. Migrate the data back to global memory
5. (on CPU) Copy data from global memory

De `__syncthreads()` functie fungeert als een soort van barrière. Alle threads in een block moeten dit punt bereiken alvorens verder te gaan, en alle geheugentoegangen van vóór de aanroep moeten zichtbaar zijn voor alle andere threads in dat block. We plaatsen dit commando dus op iedere plaats in de workflow na het kopiëren van data en voor het wegschrijven van data. Als we dit doen ziet dit er als volgt uit:

1. (on CPU) Copy data to global memory and run the kernel
2. (on GPU, per thread) Migrate one or more elements to shared memory
3. `__syncthreads()`
4. Process the data in shared memory
5. `__syncthreads()`
6. Migrate the data back to global memory
7. (on CPU) Copy data from global memory

2.0.2 Coalescing

Coalesced memory access is een optimalisatietechniek waarbij de hardware de geheugenverzoeken van alle threads in een warp combineert tot zo weinig mogelijk transacties. DRAM-geheugen is fysiek ontworpen om data te verwerken in groepen (doorgaans 32, 64 of 128 bytes). Deze aanpak maakt een parallelle toegang met hoge doorvoer mogelijk. Als we geen optimalisatie zouden toepassen verlopen de geheugenverzoeken sequentieel (per individueel verzoek), dit verminderd de prestaties aanzienlijk. Coalescing wordt bereikt wanneer de threads binnen dezelfde warp gelijktijdig opeenvolgende locaties in het globale geheugen benaderen. Wanneer dit zo is kunnen al deze individuele verzoeken gecombineerd worden tot een groot geheugenverzoeken dat gebruik

maakt van de eigenschap van DRAM om blokken geheugen te verwerken[1].
In dit labo hebben we dit geïmplementeerd en met twee specifieke kernels op matrices.

- **Copy Kernel:** Hierbij kopieert elke thread één element van matrix A naar dezelfde index in matrix B ($B = A$). Omdat opeenvolgende threads opeenvolgende kolommen kopiëren, is dit een perfect voorbeeld van coalesced geheugenverzoeken.

- **Transpose Kernel**: Threads lezen de data uit matrix A en schrijven deze weg naar de getransponeerde locaties in matrix B ($B = A^T$). Hierbij is geen coalescing mogelijk (de adressen zijn niet langer aangrenzend). Dit dus in een langere uitvoeringstijd moeten resulteren.

2.0.3 Memory types

De GPU-architectuur beschikt over verschillende soorten geheugen, elk met specifieke eigenschappen wat betreft reikwijdte (scope), locatie en snelheid.

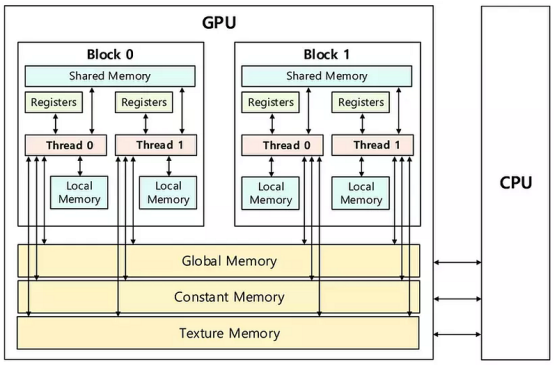


Figure 1: Geheugen types in GPU

- **Global Memory**: Dit is de standaard voor dataopslag. Het bevindt zich 'off-chip', wat resulteert in tragere toegang. Alle threads in het grid en de CPU hebben toegang tot dit geheugen, dat doorgaans meerdere gigabytes groot is.

- **Shared Memory**: Dit is een 'on-chip' geheugen (doorgaans 48kB), wat resulteert in een zeer snelle toegang. Het wordt gedeeld tussen alle threads binnen één enkel block. Zo stelt het threads in staat om data efficiënt uit te wisselen zonder telkens het trage globale geheugen aan te spreken.

- **Constant Memory**: Dit geheugen bevindt zich 'off-chip' maar is geoptimaliseerd voor low-latency read. Het is toegankelijk voor alle threads en de CPU, maar de omvang is beperkt tot 64 KB.

- **Overige types**: Daarnaast beschikt elke thread over eigen snelle **Registers** (on-chip) voor lokale variabelen en **Local Memory** (off-chip) voor wanneer de registers vol zijn. Voor specifieke 2D-grafische data kan ook **Texture Memory** worden gebruikt.

[1] Voor een optimale werking is het essentieel dat threads non-divergent zijn en effectief aangrenzende geheugenadressen aanspreken

In dit labo onderzochten we de impact van deze geheugentypes door verschillende geheugens te gebruiken voor matrix-multiplicatie ($C = A \times B$):

- **Global-only**
- **Shared Memory (Tiling)**
- **Constant Memory**

3 Observations/results

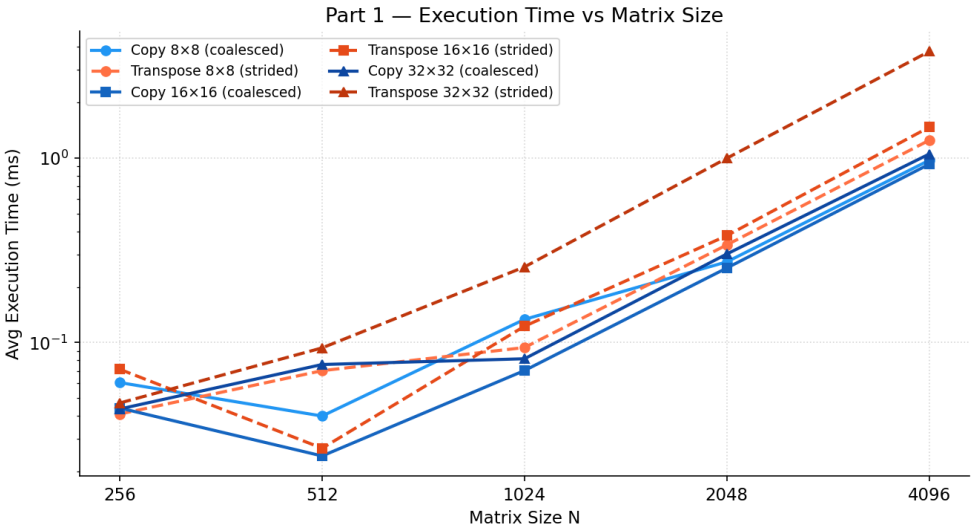


Figure 2: Execution times P1

Uit fig. 2 (en fig. 6) blijken onze vermoedens te kloppen die we eerder bespraken in. Doordat de copy kernel threads opeenvolgende kolommen kopiëren, zijn dit coalesced geheugenverzoeken. waardoor dit sneller gebeurd dan de transpose kernel.
Uit deze tijden kunnen we nu de bandbreedte BW bereken aan de hand van volgende formule

$$BW = \frac{\text{Total bytes transferred}}{\text{time}} \tag{1}$$

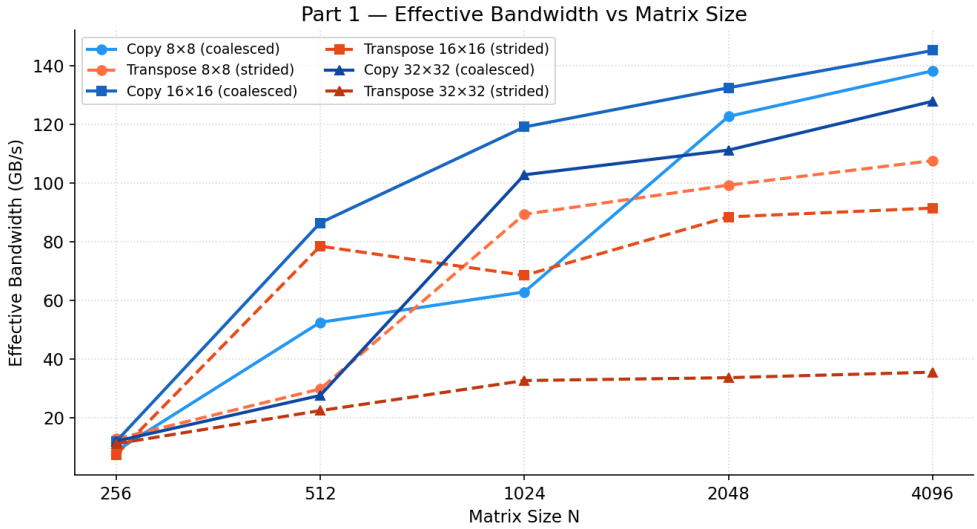


Figure 3: Effective bandwidth

Ook fig. 3 bevestigd wat we al dachten. Doordat de copy kernel beter gebruik maakt van de architectuur van het DRAM kan het ook meer van de beschikbare bandbreedte gebruiken (in vergelijking met de transpose kernel).
Bij deze resultaten is het nodig om op te merken dat de voordelen van coalescing verwateren als de matrix grote (N) verkleind. Dit is niet onlogisch. Als de grote van de matrixen kleiner wordt haalt men minder voordeel uit de 'bursty' eigenschap van het DRAM.

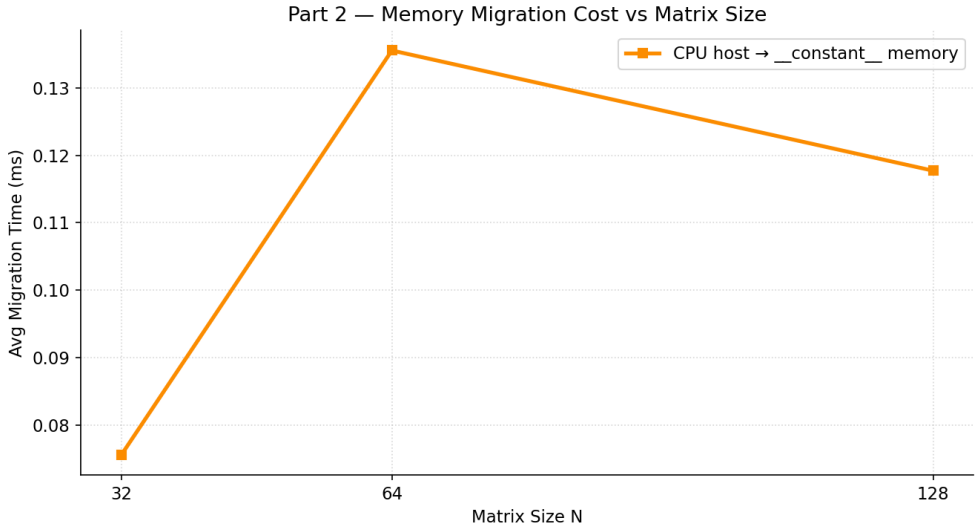


Figure 4: Migration

Op basis van de resultaten uit fig. 4 is het moeilijk om conclusies te vormen. De grafiek stelt dat er geen verband bestaat tussen de grote van de matrix en de migration time. Intuïtief zouden we

verwachten dat hoe groter de matrix, hoe groter de migration time. Dit lijkt echter niet altijd het geval te zijn.

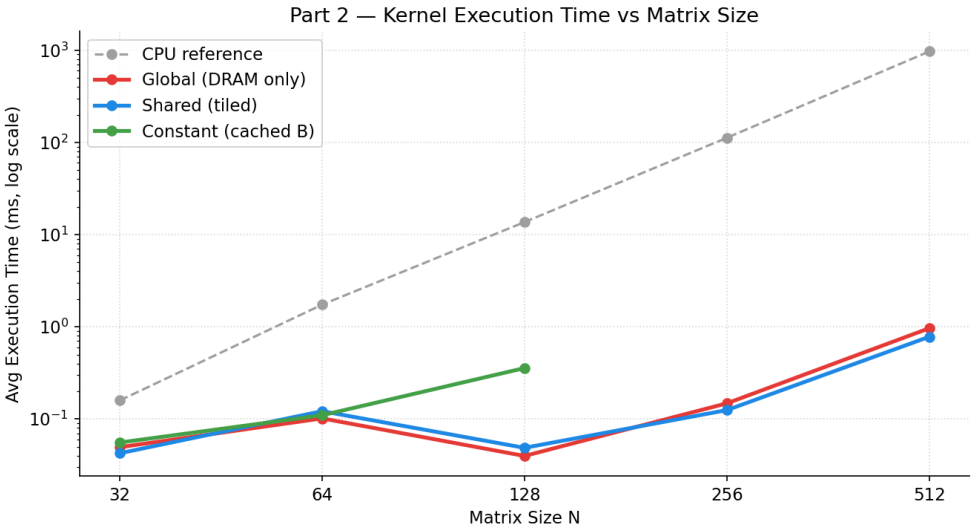


Figure 5: Execution times P2

De matrix-multiplicatie met shared memory is sneller dan de globale variant maar niet aanzienlijk sneller, beide zijn ze wel sneller dan de constant memory. Dit is eerder onverwacht. Zoals eerder vermeld in section 2.0.3 zijn het shared en constant geheugen geoptimaliseerd voor low-latency (read) in tegenstelling tot het globale geheugen.

4 Conclusion

De resultaten tonen aan dat de manier waarop geheugen wordt benaderd zeer belangrijk is. Coalesced memory access is een belangrijke techniek om de hoge latentie van geheugens te minimaliseren. Daarnaast is het ook cruciaal om de 'juiste'/'best geoptimaliseerde' geheugens aan te spreken voor specifieke opdrachten. Ten slotte is het gebruik van synchronisatie (`__syncthreads()`) essentieel bij het beheer van caches zoals shared memory om fouten te voorkomen.

A Overige grafieken

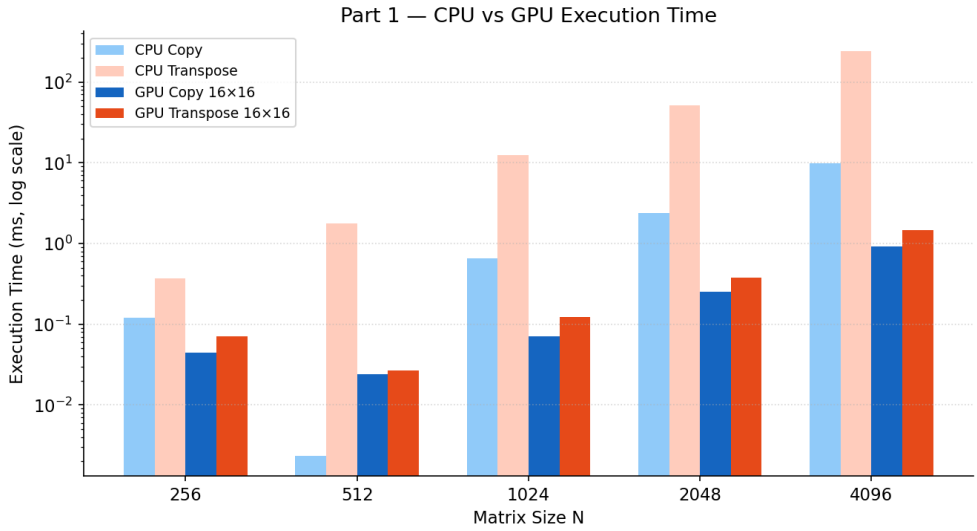


Figure 6: CPU vs GPU

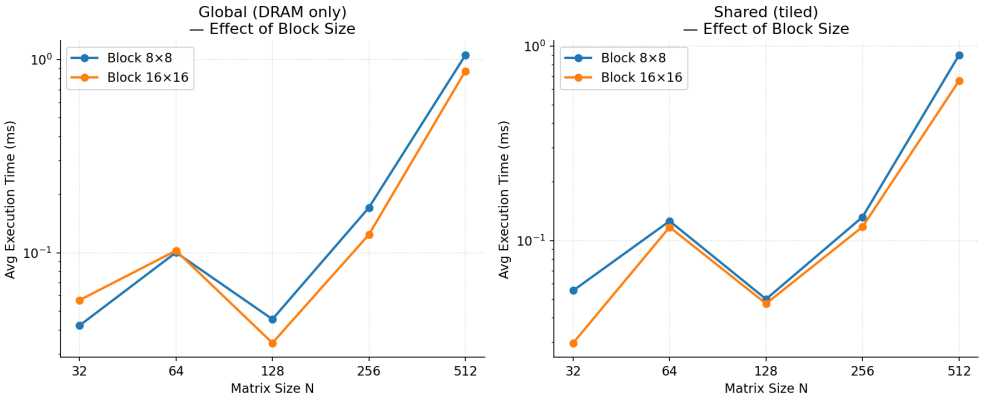


Figure 7: Blocksize P2

B Code

B.1 Part 1

```
1
2 #include <iostream>
3 #include <vector>
4 #include <cstdlib>
5 #include <cmath>
```

```cpp
#include <cuda_runtime.h>

//            CPU reference implementations


void copyMatrix_cpu(const float* A, float* B, int N) {
    for (int row = 0; row < N; row++)
        for (int col = 0; col < N; col++)
            B[row * N + col] = A[row * N + col];
}

void transposeMatrix_cpu(const float* A, float* B, int N) {
    for (int row = 0; row < N; row++)
        for (int col = 0; col < N; col++)
            B[col * N + row] = A[row * N + col];
}

//            CUDA kernels



// Copy kernel: B = A
__global__ void copyMatrix_gpu(const float* A, float* B, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N)
        B[row * N + col] = A[row * N + col];
}

// Transpose kernel: B = A^T
__global__ void transposeMatrix_gpu(const float* A, float* B, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N)
        B[col * N + row] = A[row * N + col];
}

//            Helper Functions

float avg_time_copy(const float* d_A, float* d_B, int N, dim3 grid, dim3 block,
    int runs) {
    copyMatrix_gpu<<<grid, block>>>(d_A, d_B, N); // warm-up
    cudaDeviceSynchronize();
    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);
    float total = 0.f;
    for (int r = 0; r < runs; r++) {
        cudaEventRecord(start);
        copyMatrix_gpu<<<grid, block>>>(d_A, d_B, N);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float ms = 0.f; cudaEventElapsedTime(&ms, start, stop);
        total += ms;
    }
    cudaEventDestroy(start); cudaEventDestroy(stop);
    return total / runs;
}

float avg_time_trans(const float* d_A, float* d_B, int N, dim3 grid, dim3 block,
    int runs) {
    transposeMatrix_gpu<<<grid, block>>>(d_A, d_B, N); // warm-up
    cudaDeviceSynchronize();
```

```
    cudaEvent_t start, stop;
    cudaEventCreate(&start); cudaEventCreate(&stop);
    float total = 0.f;
    for (int r = 0; r < runs; r++) {
        cudaEventRecord(start);
        transposeMatrix_gpu<<<grid, block>>>(d_A, d_B, N);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float ms = 0.f; cudaEventElapsedTime(&ms, start, stop);
        total += ms;
    }
    cudaEventDestroy(start); cudaEventDestroy(stop);
    return total / runs;
}

//           Main


int main() {
    const int NUM_RUNS = 10;

    // Vary BOTH matrix size AND block size to observe performance changes
    int matrix_sizes[] = {256, 512, 1024, 2048, 4096};
    int block_sizes[]  = {8, 16, 32};
    int num_sizes  = sizeof(matrix_sizes) / sizeof(matrix_sizes[0]);
    int num_blocks = sizeof(block_sizes)  / sizeof(block_sizes[0]);

    std::cout <<
    ================================================================\n ;
    std::cout <<  Part 1 - Coalesced Memory Access (Copy vs Transpose)\n ;
    std::cout <<  Averaged over   << NUM_RUNS <<   runs per configuration\n ;
    std::cout <<
    ================================================================\n\n ;

    for (int s = 0; s < num_sizes; s++) {
        int N = matrix_sizes[s];
        size_t sizeBytes = N * N * sizeof(float);

        std::cout <<  ==== Matrix Size N =   << N <<   ====\n ;

        // Allocate and initialize host memory once per matrix size
        std::vector<float> h_A(N * N);
        std::vector<float> h_B_copy_cpu(N * N, 0.0f);
        std::vector<float> h_B_trans_cpu(N * N, 0.0f);
        std::vector<float> h_B_gpu(N * N, 0.0f);

        for (int i = 0; i < N * N; ++i)
            h_A[i] = static_cast<float>(i % 100);

        // Run CPU reference and time it
        {
            cudaEvent_t cs, ce;
            cudaEventCreate(&cs); cudaEventCreate(&ce);

            cudaEventRecord(cs);
            copyMatrix_cpu(h_A.data(), h_B_copy_cpu.data(), N);
            cudaEventRecord(ce); cudaEventSynchronize(ce);
            float msCpuCopy = 0.f; cudaEventElapsedTime(&msCpuCopy, cs, ce);

            cudaEventRecord(cs);
            transposeMatrix_cpu(h_A.data(), h_B_trans_cpu.data(), N);
            cudaEventRecord(ce); cudaEventSynchronize(ce);
```

```
119             float msCpuTrans = 0.f; cudaEventElapsedTime(&msCpuTrans, cs, ce);
120
121             cudaEventDestroy(cs); cudaEventDestroy(ce);
122             std::cout <<    [CPU] Copy:   << msCpuCopy <<   ms   |   Transpose:   <<
     msCpuTrans <<   ms\n ;
123         }
124
125         // Allocate device memory and upload A
126         float *d_A, *d_B;
127         cudaMalloc(&d_A, sizeBytes);
128         cudaMalloc(&d_B, sizeBytes);
129         cudaMemcpy(d_A, h_A.data(), sizeBytes, cudaMemcpyHostToDevice);
130
131         double totalTransferBytes = 2.0 * sizeBytes;
132
133         // Sweep over different block sizes
134         for (int b = 0; b < num_blocks; b++) {
135             int BS = block_sizes[b];
136             dim3 block(BS, BS);
137             dim3 grid((N + BS - 1) / BS, (N + BS - 1) / BS);
138
139             float avgCopy  = avg_time_copy (d_A, d_B, N, grid, block, NUM_RUNS);
140             float avgTrans = avg_time_trans(d_A, d_B, N, grid, block, NUM_RUNS);
141
142             // Verify copy result
143             cudaMalloc(&d_B, sizeBytes);
144             copyMatrix_gpu<<<grid, block>>>(d_A, d_B, N);
145             cudaDeviceSynchronize();
146             cudaMemcpy(h_B_gpu.data(), d_B, sizeBytes, cudaMemcpyDeviceToHost);
147             bool copy_ok = true;
148             for (int i = 0; i < N * N; ++i)
149                 if (std::abs(h_B_gpu[i] - h_B_copy_cpu[i]) > 1e-5f) { copy_ok =
    false; break; }
150
151             // Verify transpose result
152             transposeMatrix_gpu<<<grid, block>>>(d_A, d_B, N);
153             cudaDeviceSynchronize();
154             cudaMemcpy(h_B_gpu.data(), d_B, sizeBytes, cudaMemcpyDeviceToHost);
155             bool trans_ok = true;
156             for (int i = 0; i < N * N; ++i)
157                 if (std::abs(h_B_gpu[i] - h_B_trans_cpu[i]) > 1e-5f) { trans_ok =
    false; break; }
158
159             double bwCopy  = (totalTransferBytes / (avgCopy  / 1000.0)) / 1e9;
160             double bwTrans = (totalTransferBytes / (avgTrans / 1000.0)) / 1e9;
161
162             std::cout <<    [Block    << BS <<  x  << BS <<  ]
163                       <<    Copy [coalesced]:     << avgCopy  <<   ms |   <<
    bwCopy  <<   GB/s |   << (copy_ok  ?  PASS  :  FAIL ) <<  \n
164                       <<
165                       <<    Transpose [strided]:   << avgTrans <<   ms |   <<
    bwTrans <<   GB/s |   << (trans_ok ?  PASS  :  FAIL ) <<  \n ;
166
167             cudaFree(d_B);
168             cudaMalloc(&d_B, sizeBytes);
169         }
170
171         cudaFree(d_A);
172         cudaFree(d_B);
173         std::cout <<  \n ;
174     }
175
```

```
176     return 0;
177 }
```

Listing 1: Cuda Kernel for exercise with Coalesced memory access

B.2 Part 2

```
1
2  #include <iostream>
3  #include <fstream>
4  #include <vector>
5  #include <cstdlib>
6  #include <cmath>
7  #include <cuda_runtime.h>
8
9  // __constant__ memory is limited to 64 KB     max 128x128 floats
10 #define MAX_CONST_N 128
11 __constant__ float d_B_const[MAX_CONST_N * MAX_CONST_N];
12
13 //           CPU reference


14 void cpu_matmul(const float* A, const float* B, float* C, int N) {
15     for (int row = 0; row < N; row++)
16         for (int col = 0; col < N; col++) {
17             float sum = 0.f;
18             for (int k = 0; k < N; k++)
19                 sum += A[row * N + k] * B[k * N + col];
20             C[row * N + col] = sum;
21         }
22 }
23
24 //           Kernel 1: Global memory only


25 __global__ void matmul_global(const float* A, const float* B, float* C, int N) {
26     int col = blockIdx.x * blockDim.x + threadIdx.x;
27     int row = blockIdx.y * blockDim.y + threadIdx.y;
28     if (row < N && col < N) {
29         float sum = 0.f;
30         for (int k = 0; k < N; k++)
31             sum += A[row * N + k] * B[k * N + col];
32         C[row * N + col] = sum;
33     }
34 }
35
36 //           Kernel 2: Shared memory (tiled)


37 // Uses BLOCK_SIZE passed as a template parameter so it works for any block size.
38 template<int BS>
39 __global__ void matmul_shared(const float* A, const float* B, float* C, int N) {
40     __shared__ float As[BS][BS];
41     __shared__ float Bs[BS][BS];
42
43     int col = blockIdx.x * BS + threadIdx.x;
44     int row = blockIdx.y * BS + threadIdx.y;
45     float sum = 0.f;
46
47     int numTiles = (N + BS - 1) / BS;
```

```
    for (int t = 0; t < numTiles; t++) {
        int a_col = t * BS + threadIdx.x;
        int b_row = t * BS + threadIdx.y;

        As[threadIdx.y][threadIdx.x] = (row < N && a_col < N) ? A[row * N + a_col] : 0.f;
        Bs[threadIdx.y][threadIdx.x] = (b_row < N && col < N) ? B[b_row * N + col] : 0.f;
        __syncthreads();

        for (int k = 0; k < BS; k++)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }

    if (row < N && col < N)
        C[row * N + col] = sum;
}

//            Kernel 3: Global + Constant memory (B in __constant__)

__global__ void matmul_constant(const float* A, float* C, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < N && col < N) {
        float sum = 0.f;
        for (int k = 0; k < N; k++)
            sum += A[row * N + k] * d_B_const[k * N + col];
        C[row * N + col] = sum;
    }
}

//            Helpers


bool verify(const float* gpu, const float* cpu, int N, float tol = 1e-2f) {
    for (int i = 0; i < N * N; i++)
        if (std::abs(gpu[i] - cpu[i]) > tol) return false;
    return true;
}

template<typename Fn>
float time_kernel(Fn fn, int warmup_runs = 1, int timed_runs = 10) {
    // warm-up
    for (int i = 0; i < warmup_runs; i++) fn();
    cudaDeviceSynchronize();

    cudaEvent_t s, e;
    cudaEventCreate(&s); cudaEventCreate(&e);
    float total = 0.f;
    for (int r = 0; r < timed_runs; r++) {
        cudaEventRecord(s);
        fn();
        cudaEventRecord(e);
        cudaEventSynchronize(e);
        float ms = 0.f; cudaEventElapsedTime(&ms, s, e);
        total += ms;
    }
    cudaEventDestroy(s); cudaEventDestroy(e);
    return total / timed_runs;
}

```

```
105 //           Main


106 int main() {
107     const int NUM_RUNS = 10;
108 
109     // Matrix sizes: constant kernel limited to N <= 128 due to __constant__ mem
110     int matrix_sizes[] = {32, 64, 128, 256, 512};
111     int block_sizes[]  = {8, 16};        // 32x32 = 1024 threads, often max
112     int num_sizes  = sizeof(matrix_sizes) / sizeof(matrix_sizes[0]);
113     int num_blocks = sizeof(block_sizes)  / sizeof(block_sizes[0]);
114 
115     std::cout <<
        ============================================================\n ;
116     std::cout <<  Part 2 - Matrix Multiplication: Global vs Shared vs Constant\n ;
117     std::cout <<  Averaged over   << NUM_RUNS <<   runs per configuration\n ;
118     std::cout <<
        ============================================================\n\n ;
119 
120     // Open CSV for Python plotting
121     std::ofstream csv( part2_results.csv );
122     csv <<  N,block_size,kernel,time_ms\n ;
123 
124     for (int s = 0; s < num_sizes; s++) {
125         int N = matrix_sizes[s];
126         size_t bytes = N * N * sizeof(float);
127 
128         std::cout <<  ==== N =   << N <<   ====\n ;
129 
130         // 1. Allocate and initialize host matrices A and B
131         std::vector<float> h_A(N * N), h_B(N * N);
132         std::vector<float> h_C_cpu(N * N), h_C_gpu(N * N);
133 
134         for (int i = 0; i < N * N; ++i) {
135             h_A[i] = static_cast<float>(rand()) / RAND_MAX;
136             h_B[i] = static_cast<float>(rand()) / RAND_MAX;
137         }
138 
139         // 2. CPU reference timing
140         float msCpu = time_kernel([&]() {
141             cpu_matmul(h_A.data(), h_B.data(), h_C_cpu.data(), N);
142         }, 0, 1);  // Just one CPU run (slow for large N)
143         std::cout <<    [CPU]             matmul:   << msCpu <<   ms\n ;
144         csv << N <<  ,0,cpu,  << msCpu <<  \n ;
145 
146         // 3. Allocate device memory
147         float *d_A, *d_B, *d_C;
148         cudaMalloc(&d_A, bytes);
149         cudaMalloc(&d_B, bytes);
150         cudaMalloc(&d_C, bytes);
151 
152         //        Memory Migration Timing: CPU     GPU (global)
153         float msH2D = time_kernel([&]() {
154             cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice);
155             cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice);
156         }, 1, NUM_RUNS);
157         std::cout <<    [Migration] CPU     GPU global:   << msH2D <<   ms (avg)\n
        ;
158 
159         // Make sure A and B are uploaded for kernel runs
160         cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice);
161         cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice);
```

```
162
163         for (int b = 0; b < num_blocks; b++) {
164             int BS = block_sizes[b];
165             dim3 block(BS, BS);
166             dim3 grid((N + BS - 1) / BS, (N + BS - 1) / BS);
167
168             std::cout <<    -- Block   << BS <<  x  << BS <<   --\n ;
169
170             //        Kernel 1: Global
171             float msGlobal = time_kernel([&]() {
172                 matmul_global<<<grid, block>>>(d_A, d_B, d_C, N);
173             }, 1, NUM_RUNS);
174             cudaMemcpy(h_C_gpu.data(), d_C, bytes, cudaMemcpyDeviceToHost);
175             bool ok_global = verify(h_C_gpu.data(), h_C_cpu.data(), N);
176             std::cout <<      Global:     << msGlobal <<   ms |   << (ok_global ?
     PASS   :  FAIL ) <<  \n ;
177             csv << N <<  ,  << BS <<  ,global,  << msGlobal <<  \n ;
178
179             //        Kernel 2: Shared (tiled)
180             float msShared = 0.f;
181             bool ok_shared = false;
182             if (BS == 8) {
183                 msShared = time_kernel([&]() {
184                     matmul_shared<8><<<grid, block>>>(d_A, d_B, d_C, N);
185                 }, 0, NUM_RUNS);
186             } else {
187                 msShared = time_kernel([&]() {
188                     matmul_shared<16><<<grid, block>>>(d_A, d_B, d_C, N);
189                 }, 1, NUM_RUNS);
190             }
191             cudaMemcpy(h_C_gpu.data(), d_C, bytes, cudaMemcpyDeviceToHost);
192             ok_shared = verify(h_C_gpu.data(), h_C_cpu.data(), N);
193             std::cout <<      Shared:     << msShared <<   ms |   << (ok_shared ?
     PASS   :  FAIL ) <<  \n ;
194             csv << N <<  ,  << BS <<  ,shared,  << msShared <<  \n ;
195
196             //        Kernel 3: Constant (only for N <= MAX_CONST_N)
197             if (N <= MAX_CONST_N) {
198                 float msGlobal2Const = time_kernel([&]() {
199                     cudaMemcpyToSymbol(d_B_const, h_B.data(), bytes);
200                 }, 1, NUM_RUNS);
201                 std::cout <<      [Migration]  g l o b a l constant :   <<
    msGlobal2Const <<   ms\n ;
202                 csv << N <<  ,  << BS <<  ,migration_to_const,  << msGlobal2Const
    <<  \n ;
203
204                 cudaMemcpyToSymbol(d_B_const, h_B.data(), bytes);
205                 float msConst = time_kernel([&]() {
206                     matmul_constant<<<grid, block>>>(d_A, d_C, N);
207                 }, 1, NUM_RUNS);
208                 cudaMemcpy(h_C_gpu.data(), d_C, bytes, cudaMemcpyDeviceToHost);
209                 bool ok_const = verify(h_C_gpu.data(), h_C_cpu.data(), N);
210                 std::cout <<      Constant:   << msConst <<   ms |   << (ok_const
    ?  PASS   :  FAIL ) <<  \n ;
211                 csv << N <<  ,  << BS <<  ,constant,  << msConst <<  \n ;
212             } else {
213                 std::cout <<      Constant: SKIPPED (N=  << N <<   > MAX_CONST_N=
     << MAX_CONST_N <<  )\n ;
214             }
215         }
216
217         cudaFree(d_A);
```

```
218         cudaFree(d_B);
219         cudaFree(d_C);
220         std::cout <<  \n ;
221     }
222
223     csv.close();
224
225     return 0;
226 }
```

Listing 2: CUDA kernels for exercise global, shared and constant memory

References

[1] Feys T. Gent (2026) *session-4*, Lab notes.